

شرح كود مشروع VAS SMS+ Monitor

هذا الملف يشرح بنية مشروع **Laravel** الموجود في `D:\vas_project` بطريقة عملية: ماذا يفعل كل جزء، كيف تتحرك الطلبات داخل التطبيق، وأين توجد الملفات المهمة التي ستتعامل معها أثناء التطوير.

الإطار: Laravel 13 مع PHP 8.3

نوع المشروع: تطبيق ويب Laravel لإدارة ومراقبة خدمات VAS/SMS

الواجهة: Blade + Bootstrap + Chart.js، مع وجود Vite/Tailwind في المشروع

قاعدة البيانات المستهدفة: Oracle عبر حزمة `yajra/laravel-oci8`

1. فكرة المشروع بشكل عام

المشروع عبارة عن لوحة مراقبة لإيرادات وخدمات VAS/SMS. يوجد تسجيل دخول، ثم يتم توجيه المستخدم إلى لوحة حسب الدور المختار أو الدور المسجل في قاعدة البيانات. الأدوار الرئيسية هي:

- **Business:** مخصص لمتابعة الإيرادات، التحليل، البحث بال MSISDN والتنبيهات.
- **Operationnel:** مخصص لمتابعة CDR، الاستيراد، التجميع، إدارة الخدمات والمزودين.
- **Admin:** مخصص لإدارة المستخدمين وبعض إعدادات النظام.

حاليا بعض الصفحات ما زالت Placeholder، بينما CRUD المستخدمين والخدمات والمزودين موجود فعليا.

2. بنية المجلدات المهمة

المسار	الدور داخل المشروع
<code>routes/web.php</code>	تعريف روابط التطبيق وصفحات الدخول وال Dashboard ومسارات Admin/Business/Operations.
<code>app/Http/Controllers</code>	المنطق الذي يستقبل الطلبات من المتصفح ويعيد Views أو redirects.
<code>app/Http/Middleware</code>	التحقق من تسجيل الدخول والصلاحيات قبل السماح بالوصول للصفحات.
<code>app/Models</code>	نماذج Eloquent التي تمثل جداول قاعدة البيانات مثل users و services و service_provider.
<code>resources/views</code>	ملفات Blade المسؤولة عن HTML والواجهة.
<code>app/Console/Commands</code>	أوامر Artisan لاستيراد ملفات CDR الخاصة بـ MMG و OCC.
<code>config/database.php</code>	إعدادات قواعد البيانات، وفيه اتصال Oracle.
<code>tests</code>	اختبارات PHPUnit الحالية.

3. كيف يتحرك الطلب داخل Laravel؟

المتصفح يطلب رابطا مثل `/dashboard` ← Laravel يبحث عن المسار في `routes/web.php` ← يمر عبر Middleware مثل `dashboard.access` ← يصل إلى Controller مثل `DashboardController@index` ← Controller يجهز البيانات ← يرجع Blade View مثل `dashboard.index` ← المتصفح يعرض الصفحة.

4. ملف المسارات routes/web.php

هذا الملف هو خريطة التطبيق. أهم ما يحتويه:

- الرابط `/` يحول المستخدم إلى `/login`.
- روابط تسجيل الدخول واستعادة كلمة المرور.
- مجموعة محمية بـ Middleware اسمها `dashboard.access`.
- داخلها مجموعات حسب الدور: `business` و `operations` و `admin`.

```
Route::middleware('dashboard.access')->group(function (): void {
    Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard');

    Route::middleware('role:operationnel')
        ->prefix('operations')
        ->name('operations.');
```

```
->group(function (): void {
    Route::resource('services', ServiceController::class)->except(['show']);
});
});
```

معنى هذا المثال: لا يمكن الوصول إلى `/dashboard` إلا بعد تسجيل الدخول أو تفعيل demo login. أما مسارات `operations/services` فتحتاج دور `operationnel`.

5. تسجيل الدخول والصلاحيات

AuthController

الملف `app/Http/Controllers/AuthController.php` مسؤول عن:

- عرض صفحة تسجيل الدخول.
- التحقق من البريد وكلمة المرور والدور المختار.
- مطابقة كلمة المرور، وإذا كانت مخزنة كنص قديم يتم تحويلها إلى Hash تلقائياً عند نجاح الدخول.
- حفظ الدور النشط داخل Session باسم `active_role`.
- دعم وضع تجربة بدون Oracle عبر `AUTH_DEMO_LOGIN=true`.
- إعادة تعيين كلمة المرور بطريقة مخصصة عن طريق البريد ورقم الهاتف.

EnsureDashboardAccess

هذا Middleware يسمح بالمرور إذا كان المستخدم مسجلاً عبر Laravel Auth أو إذا كانت جلسة demo مفعلة. إن لم يتحقق ذلك، يتم تحويل المستخدم إلى صفحة login.

EnsureUserHasRole

هذا Middleware يقرأ الدور النشط من Session أو من المستخدم، ثم يتأكد أن الدور مناسب للمسار المطلوب. إذا لم يكن مناسباً يرجع خطأ 403.

6. نموذج User

الملف `app/Models/User.php` يمثل جدول `users`. يحتوي على تعريف الأدوار وطريقة تحويل أسماء الأدوار بين شكل التطبيق وشكل Oracle.

المعنى	داخل Oracle	داخل التطبيق
مدير النظام	ADMIN	admin
محلل أعمال	BUSINESS	business
محلل عمليات	OPERATION	operationnel

الدالة `normalizeRole` مهمة لأنها تجعل التطبيق يقبل أكثر من كتابة للدور ثم يحولها إلى صيغة موحدة.

7. DashboardController

الملف `app/Http/Controllers/DashboardController.php` يعرض لوحة القيادة الرئيسية. حاليا البيانات فيه ثابتة ومكتوبة داخل الكود، مثل:

- مؤشرات KPI: إجمالي الإيرادات، عدد SMS +، عدد الخدمات، التنبيهات النشطة.
- تطور الإيرادات يوميا وشهريا.
- أفضل 5 خدمات حسب الإيرادات.
- توزيع الإيراد حسب المزود.
- قائمة تنبيهات تجريبية.

هذه البيانات يجب استبدالها لاحقا باستعلامات حقيقية من Oracle، خصوصا من جداول `ra_t_agg_mmg` و `ra_t_agg_occ`.

8. CRUD الخدمات

الملفات المهمة:

- `app/Http/Controllers/Operations/ServiceController.php`
- `app/Models/Service.php`
- `resources/views/operations/services`

هذا الجزء يسمح بعرض الخدمات، البحث، الإضافة، التعديل والحذف. يعتمد على جدول `services` ويربط كل خدمة بمزود عبر العلاقة:

```
public function provider(): BelongsTo
{
    return $this->belongsTo(VasServiceProvider::class, 'provider_id', 'id');
}
```

عند إضافة خدمة جديدة، الكود يحسب ID جديدا عن طريق: `max(id) + 1`. هذا غالبا بسبب التعامل مع Oracle بدون auto increment عادي.

9. CRUD المزودين

الملفات المهمة:

- `app/Http/Controllers/Operations/ProviderController.php`
- `app/Models/VasServiceProvider.php`
- `resources/views/operations/providers`

المزود يمثل جدول `service_provider`. العلاقة بين المزود والخدمات هي:

```
public function services(): HasMany
{
    return $this->hasMany(Service::class, 'provider_id', 'id');
}
```

عند حذف مزود، الكود يتأكد أولا أنه لا يملك خدمات مرتبطة. هذا يحمي البيانات من حذف مزود مستخدم.

10. CRUD المستخدمين

الملف `app/Http/Controllers/Admin/UserController.php` مسؤول عن إدارة المستخدمين: عرض، بحث، إضافة، تعديل وحذف.

- البريد يجب أن يكون فريدا.
- كلمة المرور يتم تخزينها باستخدام `Hash::make`.
- الأدوار تحفظ بصيغة Oracle مثل `ADMIN` و `BUSINESS`.
- لا يسمح للمستخدم بحذف حسابه الحالي.

11. نماذج الإيرادات والتنبيهات

توجد Models بسيطة تمثل الجداول التالية:

الاستخدام المتوقع	الجدول	Model
تجميع إيرادات/بيانات MMG.	ra_t_agg_mmg	RevenueAggregateMmg
تجميع إيرادات/بيانات OCC.	ra_t_agg_occ	RevenueAggregateOcc
التنبيهات أو الحوادث.	alerts	Alert

12. أوامر استيراد CDR

يوجد أمران Artisan:

- php artisan cdr:import-mmg
- php artisan cdr:import-occ

فكرة الأوامر:

- تقرأ ملفات CSV من storage/app/cdr/incoming/mmg أو storage/app/cdr/incoming/occ.
- تدخل البيانات أولاً في جدول مؤقت مثل ra_t_tmp_mmg.
- تنقل جزءاً من البيانات إلى جدول detail مثل ra_t_detail_mmg.
- إذا نجحت العملية تنقل الملف إلى processed.
- إذا فشلت تنقل الملف إلى error وتعرض رسالة الخطأ.

الأوامر تستخدم Transaction أثناء الإدخال. هذا جيد لأن أي خطأ يؤدي إلى rollback بدلا من ترك بيانات ناقصة.

13. الواجهات Blade

أهم ملف واجهة عام هو resources/views/layouts/app.blade.php. هذا الملف يحتوي على:

- هيكل الصفحة العام.
- استدعاء Bootstrap Icons و Bootstrap من CDN.
- القائمة الجانبية.
- الشريط العلوي وزر الخروج.
- CSS عام للبطاقات، الرسوم، الجداول، والاستجابة للموبايل.

القائمة الجانبية موجودة في resources/views/partials/sidebar.blade.php. هذه القائمة تتغير حسب الدور النشط للمستخدم.

14. صفحة تسجيل الدخول

الملف `resources/views/auth/login.blade.php` يعرض صفحة دخول مستقلة عن layout العام. تحتوي الصفحة على:

- اختيار نوع المستخدم: Admin أو Business أو Operationnel.
- حقول البريد وكلمة المرور.
- رابط Forget password.
- رسائل الأخطاء والنجاح.
- معلومة عن وضع demo login.

15. إعدادات قاعدة البيانات

في `config/database.php` يوجد تعريف اتصال Oracle:

```
'oracle' => [
    'driver' => 'oracle',
    'host' => env('DB_HOST', '127.0.0.1'),
    'port' => env('DB_PORT', '1521'),
    'database' => env('DB_DATABASE', ''),
    'service_name' => env('DB_SERVICE_NAME', ''),
    'username' => env('DB_USERNAME', ''),
    'password' => env('DB_PASSWORD', ''),
]
```

حتى يعمل المشروع مع Oracle يجب أن تكون قيم `.env` مضبوطة، مثلاً:

```
DB_CONNECTION=oracle
DB_HOST=...
DB_PORT=1521
DB_SERVICE_NAME=...
DB_USERNAME=...
DB_PASSWORD=...
```

16. الاختبارات الحالية

توجد اختبارات قليلة حالياً:

- `tests/Unit/ExampleTest.php`: اختبار بسيط يؤكد أن true تساوي true.
- `tests/Feature/ExampleTest.php`: يتأكد أن الرابط `/login` يحول إلى `/login`.

تم تشغيل الأمر التالي ونجح:

```
php artisan test
```

```
Tests: 2 passed, 3 assertions
```

17. ملاحظات فنية مهمة

- هناك مشكلة ترميز في بعض النصوص الفرنسية، مثل ظهور `DÃ@connexion` بدلا من `Déconnexion`.
- ملف `.env.example` ما زال افتراضيا على SQLite، بينما التطبيق يعتمد عمليا على Oracle.
- ال migrations الحالية لا تنشئ كل جداول التطبيق مثل `services` و `service_provider` و `ra_t_*`.
- Dashboard يستخدم بيانات ثابتة؛ يلزم ربطه لاحقا ببيانات Oracle الحقيقية.
- استخدام `max(id) + 1` قد يسبب مشكلة إذا أضاف مستخدمان بيانات في نفس اللحظة. الأفضل مستقبلا استخدام Oracle Sequence.
- Bootstrap و Chart.js يتم تحميلهما من CDN، وهذا يحتاج إنترنت عند تشغيل الواجهة إلا إذا تم تثبيتهما محليا.

18. خريطة العمل المقترحة لاحقا

- إصلاح ترميز النصوص الفرنسية في ملفات Blade و Controllers.
- تحديث `.env.example` لشرح إعدادات Oracle و demo login.
- ربط Dashboard بجداول `ra_t_agg_mmg` و `ra_t_agg_occ`.
- إضافة Oracle Sequences بدلا من حساب `max(id) + 1`.
- إضافة اختبارات لتسجيل الدخول والصلاحيات و CRUD.
- تحويل الصفحات Placeholder إلى صفحات فعلية حسب احتياج المشروع.

19. ملخص سريع للحفظ

إذا أردت فهم المشروع بسرعة، ابدأ من `routes/web.php` لمعرفة الروابط، ثم اقرأ Controllers لمعرفة المنطق، ثم Models لمعرفة الجداول، ثم Blade لمعرفة الواجهة. أهم منطق حالي موجود في تسجيل الدخول، الصلاحيات، CRUD المستخدمين، CRUD الخدمات، CRUD المزودين، وأوامر استيراد CDR.

تم إعداد هذا الشرح من قراءة الكود الحالي داخل المشروع. آخر تحقق: الاختبارات الحالية نجحت بأمر `php artisan test`.